UNIT 5 · TOPIC 5.5

Protecting Applications

Quiz, Answer Key

Teacher answer key, 9 questions, 1 point each. The correct answer is highlighted; the explanation and distractor notes follow.

1. Which of the following BEST describes the application security principle of "secure by design"?

○ A. Security controls are applied to an application immediately before it is released to production.

● **B. Security is incorporated into the application from the earliest stages of planning and architecture.**

○ C. All application data is encrypted at rest after the product is deployed.

○ D. Developers conduct penetration testing once a vulnerability is publicly reported.

Correct answer: B

Why: Secure by design means security is not added as an afterthought but is treated as a foundational requirement from the very start of the software development lifecycle, present in requirements, architecture, and design decisions before a line of code is written. Option B describes a late-stage practice that is reactive rather than designed-in. Option C describes one specific control (encryption at rest), not the overarching principle. Option D describes a reactive patch cycle, the opposite of proactive design.

*Why the others are wrong: **B**, Students often confuse pre-release security scanning (a good practice) with the principle of secure by design, scanning before deployment is still a late-stage, reactive measure rather than a design-phase commitment. **C**, Encryption at rest is a concrete example of a security control, but the principle of secure by design refers to the philosophy of building all such controls in from the start, not to any one control in particular. **D**, Patching after public disclosure is the defining characteristic of a reactive security posture, the opposite of what secure by design prescribes.*

2. A software company releases a new application in which all user accounts are created with minimal permissions by default, and stronger access must be explicitly granted by an administrator. Which application security principle does this BEST illustrate?

○ A. Secure by design

○ B. Defense in depth

● **C. Security by default**

○ D. Vulnerability management

Correct answer: C

Why: Security by default means the application ships in its most secure configuration without requiring the end user or administrator to harden it manually. Granting minimal permissions from account creation, the principle of least privilege as a default state, is the textbook example. Secure by design (A) refers to the development-phase practice of building security in from the start, not specifically to the runtime configuration. Defense in depth (B) refers to layering multiple independent controls. Vulnerability management (D) refers to identifying and remediating known flaws.

*Why the others are wrong: **A**, Secure by design and security by default are closely related and frequently confused; the key distinction is that secure by design is a development-lifecycle principle, while security by default describes the out-of-the-box configuration state of the shipped product. **C**, Defense in depth describes the strategy of using multiple overlapping controls at different layers, it does not specifically address default account permissions. **D**, Vulnerability management is an operational security process for tracking and patching flaws, unrelated to the default permission posture at account creation.*

3. Which of the following development activities is MOST consistent with the secure by design principle?

○ A. Running an automated vulnerability scanner against the completed application one week before launch

○ B. Issuing a security patch after users report unusual application behavior

● **C. Including threat modeling as a required step during the application planning phase**

○ D. Disabling security logging during development to improve build speed

Correct answer: C

Why: Threat modeling during planning is a foundational secure-by-design activity, it forces developers to systematically identify assets, threats, and mitigations before architecture decisions are locked in. Option A is a useful quality gate but occurs at the end of development, not by design. Option B is purely reactive. Option D actively undermines security and is the antithesis of designing security in.

*Why the others are wrong: **A**, Pre-launch scanning is better than no scanning, but it occurs after all design decisions are made, meaning any discovered vulnerability requires costly rework rather than an inexpensive design change. **B**, Responding to user-reported issues is reactive incident response, security was not considered during design. **D**, Disabling security controls for convenience violates both secure by design and security by default, introducing gaps that may persist into production.*

4. A web application allows users to enter a username in a search field. The application passes the raw user input directly into a database query without modification. Which attack does the absence of input sanitization make MOST likely, and why?

● A. A SQL injection attack, because unsanitized input can include SQL syntax that alters or exposes data in the backend database.

○ B. A brute-force attack, because the application cannot limit the rate of queries submitted by a single user.

○ C. A man-in-the-middle attack, because unencrypted input can be intercepted as it travels between the browser and server.

○ D. A denial-of-service attack, because unfiltered input can exhaust server memory by creating infinite loops in the query engine.

Correct answer: A

Why: When user input is concatenated directly into a SQL query without sanitization, an attacker can supply input such as `' OR '1'='1` or `'; DROP TABLE users; --` to manipulate the query's logic, bypass authentication, or exfiltrate the entire database. This is the definition of SQL injection. Option D (DoS via memory exhaustion) is not the primary risk of unsanitized input in this context. Option C describes a transport-layer attack, independent of whether input is sanitized. Option A (brute-force) concerns authentication rate-limiting, not input handling.

*Why the others are wrong: **A**, Brute-force and rate-limiting are authentication controls unrelated to whether input is sanitized before reaching the database. **C**, MITM attacks exploit transport-layer weaknesses (absence of TLS, certificate mismatch); they are not caused by or mitigated by input sanitization, which operates at the application layer. **D**, Server-side resource exhaustion from user input is a real concern in some contexts (e.g., regex denial-of-service, ReDoS), but it is not the primary threat when raw input is passed to a database query, SQL injection is the direct and well-documented risk.*

5. A developer configures a comment field on a blog to accept only plain text, stripping angle brackets, script tags, and all HTML before storing the input. How does this sanitization control protect other users of the application?

○ A. It protects data in transit by encoding comments before they are sent over the network.

○ B. It prevents attackers from discovering other users' session tokens by limiting the length of stored comments.

● C. It blocks stored cross-site scripting (XSS) attacks by ensuring malicious script tags cannot be embedded in content that other browsers will render.

○ D. It prevents SQL injection by removing characters that could modify a database query.

Correct answer: C

Why: Stored (persistent) XSS occurs when an attacker injects a malicious script into user-generated content that is then served to other users' browsers. By stripping HTML tags, particularly `<script>`, before storing input, the application ensures no executable JavaScript can be embedded in a comment. When other users load the page, there is no malicious payload to execute. Option C is incorrect; comment length limits do not protect session tokens. Option A conflates application-layer sanitization with transport-layer encryption. Option D is partially plausible but incorrect in context, this sanitization removes HTML, which is the right control for XSS, not the primary mechanism for SQLi defense (parameterized queries address that).

*Why the others are wrong: **A**, Transport encoding (TLS) protects data moving between client and server; application-layer sanitization protects data stored and later rendered, these are separate controls at different layers. **C**, Session token exposure is a consequence of a successful XSS attack (document.cookie theft), not a risk mitigated by comment length restrictions. **D**, Removing angle brackets does*

incidentally remove some characters used in SQLi, but the purpose here is XSS prevention; SQLi specifically requires parameterized queries or stored procedures, not HTML stripping.

6. A developer rewrites all database queries in the application to use parameterized statements (prepared statements) instead of string concatenation. How does this change protect the application from injection attacks?

○ A. The database encrypts each query before execution, making it unreadable to attackers monitoring network traffic.

○ B. The application rate-limits the number of queries any single user can submit per minute, slowing down automated attacks.

○ C. The database rejects any query that contains more than one SQL statement, blocking multi-statement injection payloads.

● D. The query structure is compiled separately from the user-supplied data, so user input can never be interpreted as executable SQL code.

Correct answer: D

Why: Parameterized queries (prepared statements) work by sending the SQL template to the database engine to be compiled first, then supplying user input as strictly typed data parameters, never as part of the SQL text. Even if an attacker enters `'; DROP TABLE users; --`, the database treats it as a literal string value, not as SQL to execute. This completely eliminates the injection surface for that query. Option A is incorrect; parameterization is not encryption. Option D describes rate-limiting, a separate control. Option C describes a specific database configuration option, not what parameterized queries do.

*Why the others are wrong: **A**, Parameterization operates at the query-building layer, not the transport or storage encryption layer; the two are independent and complementary controls. **C**, Rejecting multi-statement queries is one configuration mitigation, but it can be bypassed by single-statement injections (e.g., `OR 1=1`); parameterized queries prevent all injection regardless of statement count. **D**, Rate-limiting reduces the window for brute-force or automated attack campaigns but does not prevent a single well-crafted injection payload from succeeding on the first attempt.*

7. A security review of an e-commerce application reveals that product review fields render raw HTML entered by users directly in other customers' browsers. The development lead argues the issue should be addressed in a future release to avoid delaying the current launch. Evaluate this decision from the perspective of application security principles.

○ A. The decision is acceptable because stored XSS is a low-severity issue that primarily affects only the attacker's own browser session.

● B. The decision is problematic because it exposes every user who views a poisoned review to malicious script execution, and it reflects a failure of both secure by design and security by default principles.

○ C. The decision is acceptable because HTTPS encryption will prevent injected scripts embedded in page content from executing in users' browsers.

○ D. The decision is problematic only if the application stores payment card data, since PCI-DSS would mandate an immediate fix.

Correct answer: B

Why: Stored XSS is a high-severity vulnerability ranked in the OWASP Top 10. A script injected into a review is served to every user who loads that page, enabling session hijacking (via document.cookie theft), credential harvesting through fake login overlays, drive-by malware distribution, and defacement, all without any further interaction from the attacker. Delaying the fix also reveals two compounding design failures: (1) the application was not built with secure by design practices (sanitization was never architected in), and (2) it does not meet security by default (it allows unsafe HTML through without user or admin action to restrict it). Option A is factually wrong, stored XSS affects every visitor, not just the attacker. Option C is wrong, HTTPS protects data in transit; it has no effect on scripts already embedded in the served page content. Option D incorrectly scopes the risk to payment data only, XSS can be exploited regardless of what data the application handles.

*Why the others are wrong: **A**, Students sometimes conflate reflected XSS (which does primarily affect a targeted user who clicks a crafted link) with stored XSS; stored XSS is persistent and affects all users who view the infected page, making it higher severity. **C**, HTTPS is a transport-layer control; it protects data moving between the browser and server but cannot remove or neutralize a script that the server has already embedded in a legitimate HTML response. **D**, While PCI-DSS compliance is a real obligation for applications handling card data, application security obligations, and ethical duties to users, exist independently of regulatory scope; any user's session is at risk from XSS.*

8. Two development teams are building separate patient data portals. Team Alpha conducts threat modeling and embeds input validation controls during the architecture phase. Team Beta builds the full UI first and then adds input validation and access controls in the final testing sprint. Both applications pass the same pre-launch security scan with zero critical findings. Which evaluation of the two approaches is MOST accurate from an application security standpoint?

● A. Team Alpha's approach is more sound because security controls architected from the start are less likely to contain design-level flaws that automated scanners cannot detect, and remediation costs are far lower when issues are caught during design.

○ B. Team Alpha's approach is preferable only in regulated industries; for general applications the approaches carry equivalent risk.

○ C. Team Beta's approach is more efficient because it avoids slowing down the development process with security considerations that can be validated at the end.

○ D. Both approaches are equally secure because they produced the same scan results, and the timing of security activities does not affect the final security posture.

Correct answer: A

Why: Automated vulnerability scanners detect known patterns in compiled or running code; they cannot evaluate design-level decisions such as whether threat modeling was performed, whether trust boundaries are correctly placed, or whether business logic flaws exist. Team Alpha's threat-modeling step surfaces risks that are cheap to resolve at design time, NIST's Secure Software Development Framework (SSDF) and research by NIST/IBM both document that a vulnerability costs orders of magnitude more to fix post-deployment than during design. Team Beta's late-stage controls may also carry architectural compromises: controls bolted onto a completed design are often incomplete or bypassable because the underlying design was never reviewed for security assumptions. The identical scan results do not imply identical security, they imply identical scanner coverage, which is a subset of total security posture.

*Why the others are wrong: **A**, OWASP's SAMM and NIST's SSDF apply universally; the principles of secure by design and security by default reduce risk for all application types, not only regulated ones. **C**, The efficiency argument ignores the well-documented cost curve of late remediation; a vulnerability found during design costs minutes to fix, whereas the same vulnerability found post-deployment may require significant rearchitecting, re-testing, and re-deployment. **D**, This conflates scanner pass rate with security equivalence; scanners have limited coverage and cannot find logic flaws, architectural weaknesses, or missing controls that were never designed in.*

9. A penetration tester discovers that a file-management application accepts a target file path directly from user input and passes it to a server-side file-read operation with no sanitization or path restriction. The application owner argues that because all users must be authenticated, the risk is low and remediation can be deferred. Evaluate this reasoning and identify the correct remediation approach.

○ A. The reasoning is sound; authentication ensures that only authorized users access the system, which sufficiently limits the impact of unsanitized file paths.

○ B. The reasoning is flawed, but the correct remediation is to replace the file system with a database, eliminating direct file-path handling entirely.

○ C. The reasoning is sound; path traversal is only exploitable over unauthenticated connections, and the finding should be marked as informational rather than a vulnerability.

● D. The reasoning is flawed; authentication does not prevent an authenticated user from supplying traversal sequences such as ../../etc/shadow to read files outside the intended directory, and remediation requires canonicalizing and allowlist-validating the resolved path before any file operation.

Correct answer: D

Why: Path traversal (CWE-22, OWASP Top 10 category A01: Broken Access Control) allows an attacker to use sequences like `../` to navigate outside the intended directory and read or overwrite sensitive server files, including credentials, configuration files, or private keys. Authentication confirms identity but does not constrain what file paths an authenticated user can supply. A legitimate user can be the attacker, or a legitimate user's session can be hijacked. Correct remediation involves: (1) stripping or rejecting path traversal sequences before processing, (2) resolving the full canonical path using the OS path library, and (3) verifying the canonical path starts with an approved base directory before performing any file operation. Option A is incorrect: authentication and authorization are different controls, authorization must also govern which paths are accessible. Option C is factually wrong: authenticated attackers are a well-documented threat model (insider threat, compromised accounts). Option B is disproportionate and does not address the sanitization gap, even database-backed systems can be vulnerable to injection if input is not sanitized.

*Why the others are wrong: **A**, Students often treat authentication as a general security guarantee; it is not, it establishes identity, but a malicious or compromised authenticated user can still exploit application-layer vulnerabilities like path traversal. **B**, While database-backed designs can simplify some path-handling concerns, the root cause is unsanitized input, not the storage medium; the fix is input sanitization and path validation, not a full system replacement. **C**, This reflects a common misconception about threat models; insider threats, session hijacking, and privilege-escalation scenarios all involve authenticated actors, making authentication an insufficient boundary for access control decisions.*